

C-programming 2023 Solution

Group-A(2 marks questions)

1

A) Define tail recursion?

Ans) Tail recursion refers to a situation where a function makes a recursive call as its last operation before returning a result. This allows the compiler to optimize the recursion by reusing the current function's stack frame for the next function call, potentially transforming the recursion into an iterative loop, thus saving memory and avoiding stack overflow for deep recursion.

B) What do you mean by `int *a[20]` ?

Ans) `int *a[20]` declares an array named 'a' consisting of 20 elements, whereas 'int *' shows that each element is a pointer to an integer. This means each element in the array can store the address of an integer, allowing for dynamic memory allocation and manipulation of integer values through these pointers.

C) State the characteristics of an algorithm.

Ans) An algorithm in C programming has the following key characteristics:

1. **Definiteness:** Each step is clearly and precisely defined.
2. **Input:** It has zero or more inputs.
3. **Output:** It produces one or more outputs.
4. **Finiteness:** It terminates after a finite number of steps.
5. **Effectiveness:** The steps are simple and can be performed.
6. **Correctness:** It solves the problem correctly.
7. **Efficiency:** It makes good use of time and space resources.

D) How two dimensional arrays is declared and initialized?

Ans) Declaration: To declare a two-dimensional array, you specify the type of elements, the array name, and the number of elements in each dimension.

Initialization: You can initialize a two-dimensional array at the time of declaration using nested braces for the rows and columns.

E) What is generic pointer?

Ans) A generic pointer, also known as a void pointer in C programming, is a special type of pointer that can point to objects of any data type. It is declared using the 'void' keyword. Unlike other pointers, a void pointer does not have a specific data type associated with the memory address it points to.

Group-B(6 marks questions)

2) Explain logical operators with example.

Ans) Logical operators in C programming are used to perform logical operations on one or more conditions or expressions. They are typically used in control statements like `if`, `while`, and `for` to combine multiple conditions.

Logical Operators:

1. Logical AND (`&&`):

- This operator returns `true` (non-zero) if both operands are true, and `false` (zero) otherwise.
- Example:

```
c Copy code  
  
int a = 5, b = 10, c = 15;  
if (a < b && b < c) {  
    printf("Both conditions are true.\n");  
}
```

- Here, `a < b` and `b < c` are both true, so the output will be "Both conditions are true."

2. Logical OR (`||`):

- This operator returns `true` (non-zero) if at least one of the operands is true, and `false` (zero) if both are false.
- Example:

```
c Copy code  
  
int a = 5, b = 10, c = 15;  
if (a > b || b < c) {  
    printf("At least one condition is true.\n");  
}
```

- Here, `a > b` is false, but `b < c` is true, so the output will be "At least one condition is true."

3. Logical NOT (`!`):

- This operator reverses the logical state of its operand. If the condition is true, it returns false, and if it is false, it returns true.
- Example:

```
c Copy code  
  
int a = 5, b = 10;  
if (!(a > b)) {  
    printf("The condition is false.\n");  
}
```

- Here, `a > b` is false, so `!(a > b)` is true, and the output will be "The condition is false."

3) Write short note on `sprint()` , `putc()` , `getche()`.

Ans)

`getche()`

The `getche()` function reads a character from the console and echoes it back to the screen. It is part of the `conio.h` library, which is available on some systems, such as DOS and Windows, but not standard C libraries.

Syntax:

```
c Copy code  
  
int getche(void);
```

`putc()`

The `putc()` function writes a character to a specified output stream. It is similar to `fputc()`, but can be implemented as a macro in some systems, leading to slight differences in behavior.

Syntax:

```
c Copy code  
  
int putc(int char, FILE *stream);
```

``getche()``

The ``getche()`` function reads a character from the console and echoes it back to the screen. It is part of the ``conio.h`` library, which is available on some systems, such as DOS and Windows, but not standard C libraries.

Syntax:

```
c Copy code  
  
int getche(void);
```

4) What is function? Discuss call by reference with example.(2+4=6 marks)

Ans)

A function in C is a self-contained block of code designed to perform a specific task. It takes input in the form of parameters, processes these inputs, and returns a result. Functions help in organizing code, making it more modular, reusable, and easier to debug and maintain.

Function Syntax:

```
c Copy code  
  
return_type function_name(parameter_list) {  
    // Function body  
    // Code to be executed  
}
```

Call by Reference

Call by reference is a method of passing arguments to a function where the function receives the reference (address) of the variable, rather than a copy of its value. This means any changes made to the parameter inside the function affect the original variable.

Example:

Here's an example to illustrate call by reference in C:

```
c Copy code

#include <stdio.h>

// Function to swap the values of two integers
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

int main() {
    int a = 5, b = 10;

    printf("Before swap: a = %d, b = %d\n", a, b);

    // Call by reference
    swap(&a, &b);

    printf("After swap: a = %d, b = %d\n", a, b);

    return 0;
}
```

5) What is null pointer? Define and declare pointers with example.(2+4=6marks)

Ans) A null pointer in C is a pointer that does not point to any valid memory location. It is used to signify that the pointer is intentionally not assigned to any object or data. A null pointer is particularly useful for indicating that a pointer is not intended to reference any memory location, for error checking, or for initializing pointers.

A pointer in C is a variable that stores the memory address of another variable. Pointers are a powerful feature in C that allow for dynamic memory allocation, efficient array handling, and the creation of complex data structures such as linked lists and trees.

Pointer Definition and Declaration

Syntax:

To define and declare a pointer, use the asterisk `\*` before the pointer name during declaration.

```
c
type *pointer_name;
```

- `type`: The data type of the variable that the pointer will point to.
- `pointer_name`: The name of the pointer variable.

Example:

Here's an example demonstrating the definition, declaration, initialization, and usage of pointers.

```
c
#include <stdio.h>

int main() {
    // Declare an integer variable
    int var = 10;

    // Declare a pointer to an integer
    int *ptr;

    // Initialize the pointer with the address of var
    ptr = &var;

    // Output the value of var directly
    printf("The value of var: %d\n", var);

    // Output the value of var using the pointer
    printf("The value of var using pointer: %d\n", *ptr);

    // Output the address of var
    printf("The address of var: %p\n", (void*)&var);

    // Output the address stored in pointer ptr
    printf("The address stored in pointer ptr: %p\n", (void*)ptr);

    return 0;
}
```

6) Difference between compiler and interpreter. What is the role of linker and loader?(3+3=6marks)

Ans)

Feature	Compiler	Interpreter
Translation	Translates entire source code at once	Translates and executes source code line-by-line
Execution	Executes the compiled machine code	Executes the source code directly
Speed	Faster execution after compilation	Slower execution due to simultaneous translation
Error Detection	Errors detected at compile-time	Errors detected at runtime
Output	Generates an executable file	Does not produce an executable file
Languages	Examples: C, C++, Java (compiles to bytecode)	Examples: Python, JavaScript, Ruby
Usage	Suitable for performance-critical applications	Suitable for scripting and rapid development
Error Handling	All errors must be fixed before running	Errors are handled one line at a time

Linker

The linker is a tool that combines multiple object files into a single executable file and resolves references between them. Its roles are as follows-

1. **Combining Object Files:** Merges multiple object files generated by the compiler into a single executable file.
2. **Symbol Resolution:** Resolves references to undefined symbols by linking them with their corresponding definitions across different object files.
3. **Address Binding:** Assigns final memory addresses to code and data sections, determining their locations in memory.
4. **Library Linking:** Links the program with necessary libraries, incorporating additional code that the program depends on.
5. **Relocation:** Adjusts addresses within the code and data sections to ensure that all references are correctly resolved.

Loader

The loader is part of the operating system and is responsible for loading the executable file into memory and preparing it for execution. . Its roles are as follows-

1. **Loading**: Loads the executable file from disk into memory.
2. **Memory Allocation**: Allocates memory for the program's code, data, and stack.
3. **Address Binding**: Resolves any addresses that are left unresolved by the linker, such as dynamic library references.
4. **Program Start**: Initializes the program's runtime environment and starts program execution by transferring control to the program's entry point (usually the `main` function in C).

7) Consider an array `int marks[20][5]` where base address=1000 and number of words per memory location of the array=2. Compute the address of `marks[10][4]` in row major and column major order.(3+3=6marks)

Ans)

Given:

- Array: `int marks[20][5]`
- Base address (BA): 1000
- Number of words per memory location (word size): 2

Row-Major Order

In row-major order, the elements of the array are stored row by row. The address of an element `marks[i][j]` is calculated as follows:

$$\text{Address}(\text{marks}[i][j]) = \text{Base Address} + [(i \times \text{number of columns} + j) \times \text{word size}]$$

For `marks[10][4]`:

1. Base Address (BA): 1000
2. Number of columns (C): 5
3. Word size (W): 2
4. Row index (i): 10
5. Column index (j): 4

Substituting the values:

$$\text{Address}(\text{marks}[10][4]) = 1000 + [(10 \times 5 + 4) \times 2]$$

$$\text{Address}(\text{marks}[10][4]) = 1000 + [(50 + 4) \times 2]$$

$$\text{Address}(\text{marks}[10][4]) = 1000 + [54 \times 2]$$

$$\text{Address}(\text{marks}[10][4]) = 1000 + 108$$

$$\text{Address}(\text{marks}[10][4]) = 1108$$

Column-Major Order

In column-major order, the elements of the array are stored column by column. The address of an element `marks[i][j]` is calculated as follows:

$$\text{Address}(\text{marks}[i][j]) = \text{Base Address} + [(j \times \text{number of rows} + i) \times \text{word size}]$$

For `marks[10][4]`:

1. Base Address (BA): 1000
2. Number of rows (R): 20
3. Word size (W): 2
4. Row index (i): 10
5. Column index (j): 4

Substituting the values:

$$\text{Address}(\text{marks}[10][4]) = 1000 + [(4 \times 20 + 10) \times 2]$$

$$\text{Address}(\text{marks}[10][4]) = 1000 + [(80 + 10) \times 2]$$

$$\text{Address}(\text{marks}[10][4]) = 1000 + [90 \times 2]$$

$$\text{Address}(\text{marks}[10][4]) = 1000 + 180$$

$$\text{Address}(\text{marks}[10][4]) = 1180$$

Summary

- Row-Major Order: The address of `marks[10][4]` is `1108`.
- Column-Major Order: The address of `marks[10][4]` is `1180`.

Group-C(10 marks questions)

8) Define recursive function. Write a program using recursion to calculate Fibonacci series – 0 1 1 2 3 5 8 13 21.....(2+8=10 marks)

Ans)

Recursive Function

A recursive function is a function that calls itself directly or indirectly in order to solve a problem. It solves a problem by breaking it down into smaller instances of the same problem until it reaches the simplest form, known as the base case, which can be solved directly.

Key Characteristics:

1. **Base Case:** The condition under which the function stops calling itself, preventing infinite recursion. This is typically the simplest case that can be handled without further recursion.
2. **Recursive Case:** The part of the function that includes the function calling itself with modified arguments, progressively working towards the base case.

C program that calculates the Fibonacci series using recursion:

```
c Copy code

#include <stdio.h>

// Recursive function to calculate Fibonacci number
int fibonacci(int n) {
    if (n == 0) {
        return 0;
    } else if (n == 1) {
        return 1;
    } else {
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}

int main() {
    int n, i;

    printf("Enter the number of terms in the Fibonacci series: ");
    scanf("%d", &n);

    printf("Fibonacci series: ");
    for (i = 0; i < n; i++) {
        printf("%d ", fibonacci(i));
    }

    return 0;
}
```

9) What is nested structure? Write a program to read and display the information(name, emp no, address and phone no.) of 10 employees.(2+8=10 marks)

Ans) A nested structure in C refers to a structure definition that is declared within another structure definition. This allows for the creation of complex data structures where one structure contains another as a member.

Syntax of nested structure:

```
c Copy code

struct OuterStructure {
    // Members of the outer structure
    // Can include other data types or structures

    struct InnerStructure {
        // Members of the inner structure
        // Can include variables of any data type
    } inner; // Optional name for the instance of the inner structure
};
```

Code:

```
#include <stdio.h>
#include <string.h>

// Define a structure for employee information
struct Employee {
    char name[50];
    int emp_no;
    char address[100];
    char phone_no[15];
};

int main() {
    struct Employee employees[10]; // Array to store information of 10 employees
    int i;

    // Reading employee information
    for (i = 0; i < 10; i++) {
        printf("Enter information for employee %d:\n", i + 1);

        printf("Name: ");
        fgets(employees[i].name, 50, stdin);
        // Remove newline character from the end of the string
        employees[i].name[strcspn(employees[i].name, "\n")] = '\0';

        printf("Employee Number: ");
        scanf("%d", &employees[i].emp_no);
        getchar(); // To consume the newline character left by scanf
    }
}
```

```

    printf("Address: ");
    fgets(employees[i].address, 100, stdin);
    // Remove newline character from the end of the string
    employees[i].address[strcspn(employees[i].address, "\n")] = '\0';

    printf("Phone Number: ");
    fgets(employees[i].phone_no, 15, stdin);
    // Remove newline character from the end of the string
    employees[i].phone_no[strcspn(employees[i].phone_no, "\n")] = '\0';

    printf("\n");
}

// Displaying employee information
printf("Employee Information:\n");
for (i = 0; i < 10; i++) {
    printf("Employee %d:\n", i + 1);
    printf("Name: %s\n", employees[i].name);
    printf("Employee Number: %d\n", employees[i].emp_no);
    printf("Address: %s\n", employees[i].address);
    printf("Phone Number: %s\n", employees[i].phone_no);
    printf("\n");
}

return 0;
}

```

Sample Output:

yaml

 Copy code

Enter information for employee 1:

Name: John Doe

Employee Number: 12345

Address: 123 Elm Street

Phone Number: 555-1234

...

Employee Information:

Employee 1:

Name: John Doe

Employee Number: 12345

Address: 123 Elm Street

Phone Number: 555-1234

...

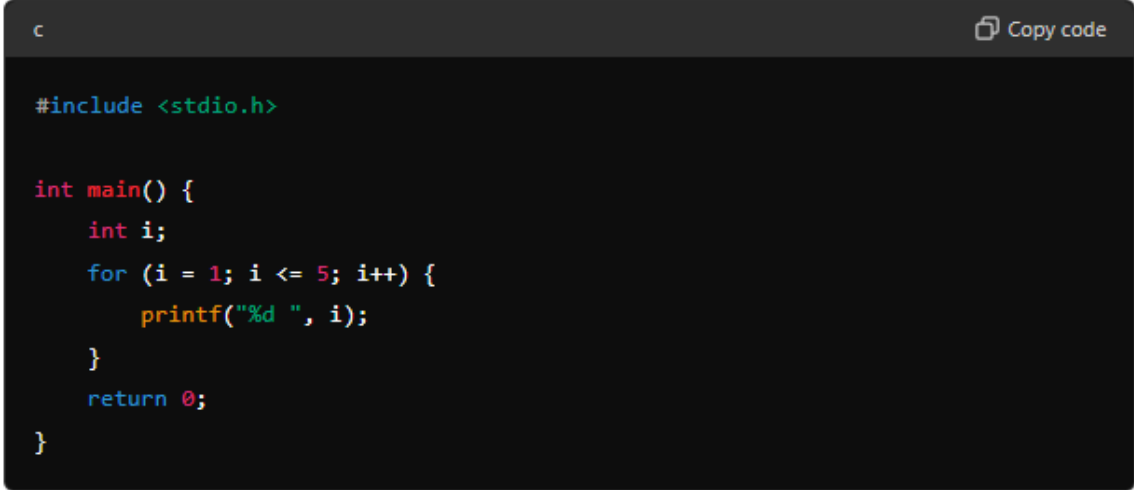
10) State importance of header file. Explain different types of loops with example in C.(2+8=10 marks)

Ans) Header files in C are essential for organizing code and making it easier to understand and maintain. They contain declarations of functions, variables, and data structures used throughout a program, promoting code reuse and readability. By encapsulating interface details, header files hide implementation complexities, simplifying how modules interact. They also provide documentation and prevent repetitive file inclusions, ensuring smooth compilation and integration of code components.

In C programming, there are three main types of loops: **for**, **while**, and **do-while**. Each loop type has its own syntax and use cases. Here's an explanation of each with examples:

1. For Loop:

- **Definition:** The for loop is a control flow statement that allows you to execute a block of code repeatedly based on a condition. It consists of three parts: initialization, condition, and update.
- **Example:**



```
c Copy code  
  
#include <stdio.h>  
  
int main() {  
    int i;  
    for (i = 1; i <= 5; i++) {  
        printf("%d ", i);  
    }  
    return 0;  
}
```

- **Explanation:** In this example, the for loop initializes the variable `i` to 1, executes the loop body while `i` is less than or equal to 5, and increments `i` by 1 in each iteration. It prints the value of `i` in each iteration, resulting in the output: `1 2 3 4 5`.

2. While Loop:

- **Definition:** The while loop is a control flow statement that repeatedly executes a block of code as long as the specified condition is true.
- **Example:**

```
c Copy code  
  
#include <stdio.h>  
  
int main() {  
    int i = 1;  
    while (i <= 5) {  
        printf("%d ", i);  
        i++;  
    }  
    return 0;  
}
```

- **Explanation:** In this example, the while loop checks the condition `i <= 5` before each iteration. If the condition is true, it executes the loop body, increments `i`, and repeats the process until the condition becomes false. It prints the value of `i` in each iteration, resulting in the output: `1 2 3 4 5`.

3. Do-While Loop:

- **Definition:** The do-while loop is a control flow statement that executes a block of code at least once, and then repeatedly executes the block as long as the specified condition is true.
- **Example:**

```
c Copy code  
  
#include <stdio.h>  
  
int main() {  
    int i = 1;  
    do {  
        printf("%d ", i);  
        i++;  
    } while (i <= 5);  
    return 0;  
}
```

- **Explanation:** In this example, the do-while loop executes the loop body at least once, regardless of the condition `i <= 5`. After each iteration, it checks the condition. If the condition is true, it continues executing the loop body and increments `i`. It prints the value of `i` in each iteration, resulting in the output: `1 2 3 4 5`.

11) What is file? Write a program to create a file. Write in it and close File(2+8=10marks)

Ans) In computing, a file is a named collection of data or information that is stored on a storage device, such as a hard disk, solid-state drive, or optical disc. Files are used to organize and store data in a structured manner for later retrieval and processing. In addition to storing data, files can also contain metadata, such as the file's name, size, type, and permissions, which provide information about the file and how it can be accessed or manipulated.

Here's a simple C program that creates a file, writes content into it, and then closes the file:

c

 Copy code

```
#include <stdio.h>

int main() {
    FILE *filePointer; // Declare a file pointer

    // Open file in write mode
    filePointer = fopen("example.txt", "w");

    // Check if file opened successfully
    if (filePointer == NULL) {
        printf("Unable to create file.\n");
        return 1;
    }

    // Write content into the file
    fprintf(filePointer, "This is some text written into the file.");

    // Close the file
    fclose(filePointer);

    printf("File created, written, and closed successfully.\n");

    return 0;
}
```

12) Differentiate between array and structure. Define union and how members of a union are accessed using a program code.(3+2+5=10 marks)

Ans)

Aspect	Array	Structure
Definition	Collection of elements of the same data type, stored in contiguous memory locations	Composite data type that groups together different types of data elements under a single name
Data Organization	Linear sequence accessed by index	Grouping of data elements with unique names
Size	Fixed at compile time	Determined by sum of member sizes
Memory Allocation	Contiguous block allocation	Separate allocation for each member
Usage	Suitable for homogeneous data	Suitable for heterogeneous data

A union in C is a user-defined data type that allows you to store different types of data in the same memory location. Unlike structures, where each member has its own memory space, all members of a union share the same memory location. This means that only one member of the union can be active or valid at any given time. The size of a union is determined by the size of its largest member.

Here's a program demonstrating how members of a union are accessed:

c

 Copy code

```
#include <stdio.h>

// Define a union named Data
union Data {
    int intValue;
    float floatValue;
    char stringValue[20];
};

int main() {
    union Data data; // Declare a variable of type union Data

    // Accessing members of the union
    data.intValue = 10; // Assigning value to intValue
    printf("intValue: %d\n", data.intValue);

    data.floatValue = 3.14; // Assigning value to floatValue
    printf("floatValue: %.2f\n", data.floatValue);

    strcpy(data.stringValue, "Hello"); // Assigning value to stringValue
    printf("stringValue: %s\n", data.stringValue);

    return 0;
}
```